

PRAKTIKUM SISTEM OPERASI

MODUL 6

PROSES SEKUENSIAL DAN KONKUREN



Oleh:

Rizaldy Aulia Rachman 2311104051

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

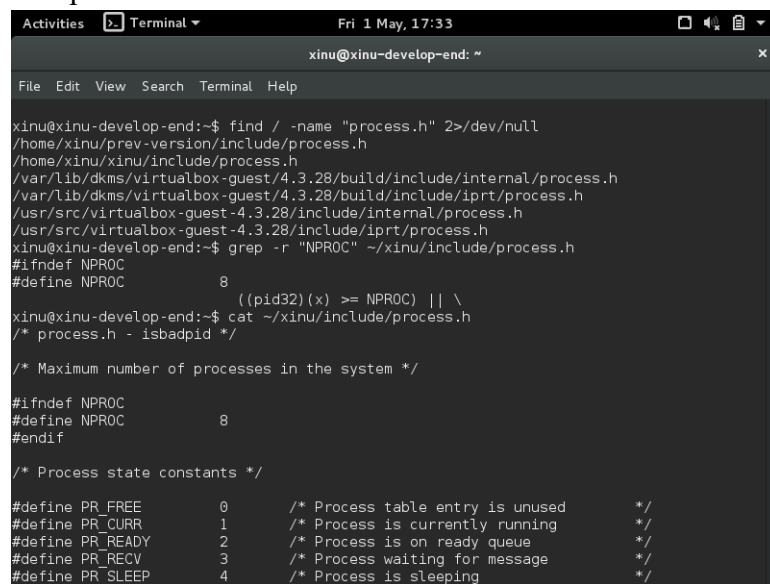
UNIVERSITAS TELKOM

2026

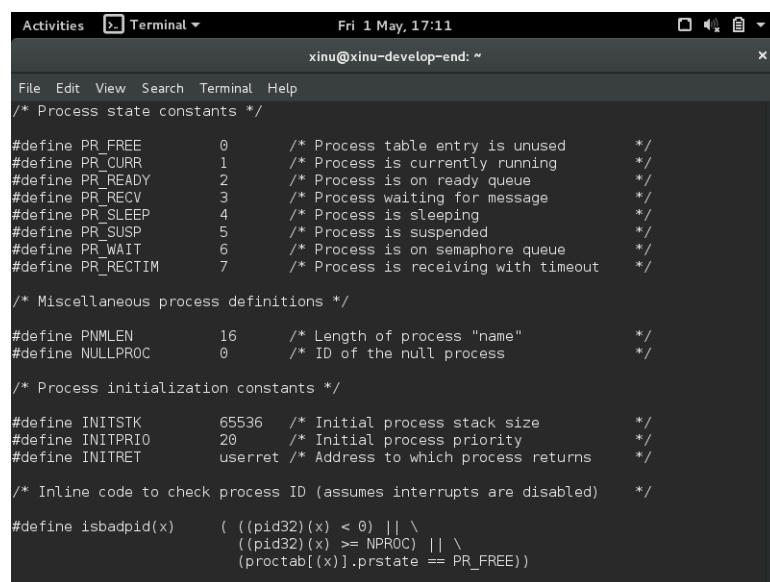
JURNAL/TP

1. Selain dipengaruhi oleh kapasitas perangkat keras (memori), jumlah maksimum proses juga dapat dibatasi melalui perangkat lunak. Pada sistem Linux, batas maksimal proses adalah 4.194.303 untuk arsitektur 64-bit dan 32.767 untuk 32-bit. Nilai tersebut dapat diperiksa menggunakan perintah `cat /proc/sys/kernel/pid_max`.

- File `process.h`



```
xinu@xinu-develop-end: ~  
Fri 1 May, 17:33  
File Edit View Search Terminal Help  
  
xinu@xinu-develop-end:~$ find / -name "process.h" 2>/dev/null  
/home/xinu/prev-version/include/process.h  
/home/xinu/xinu/include/process.h  
/var/lib/dkms/virtualbox-guest/4.3.28/build/include/internal/process.h  
/var/lib/dkms/virtualbox-guest/4.3.28/build/include/iprt/process.h  
/usr/src/virtualbox-guest-4.3.28/include/internal/process.h  
/usr/src/virtualbox-guest-4.3.28/include/iprt/process.h  
xinu@xinu-develop-end:~$ grep -r "NPROC" ~/xinu/include/process.h  
#ifndef NPROC  
#define NPROC      8  
      ((pid32)(x) >= NPROC) || \  
xinu@xinu-develop-end:~$ cat ~/xinu/include/process.h  
/* process.h - isbadpid */  
  
/* Maximum number of processes in the system */  
  
#ifndef NPROC  
#define NPROC      8  
#endif  
  
/* Process state constants */  
  
#define PR_FREE      0      /* Process table entry is unused      */  
#define PR_CURR      1      /* Process is currently running      */  
#define PR_READY     2      /* Process is on ready queue         */  
#define PR_RECV      3      /* Process waiting for message       */  
#define PR_SLEEP     4      /* Process is sleeping               */  
#define PR_SLEEP     4      /* Process is sleeping               */
```



```
xinu@xinu-develop-end: ~  
Fri 1 May, 17:11  
File Edit View Search Terminal Help  
  
/* Process state constants */  
  
#define PR_FREE      0      /* Process table entry is unused      */  
#define PR_CURR      1      /* Process is currently running      */  
#define PR_READY     2      /* Process is on ready queue         */  
#define PR_RECV      3      /* Process waiting for message       */  
#define PR_SLEEP     4      /* Process is sleeping               */  
#define PR_SLEEP     4      /* Process is sleeping               */  
#define PR_SUSP      5      /* Process is suspended              */  
#define PR_WAIT      6      /* Process is on semaphore queue     */  
#define PR_RECTIM    7      /* Process is receiving with timeout */  
  
/* Miscellaneous process definitions */  
  
#define PNMLEN      16      /* Length of process "name"          */  
#define NULLPROC    0      /* ID of the null process             */  
  
/* Process initialization constants */  
  
#define INITSTK      65536  /* Initial process stack size        */  
#define INITPRIO     20     /* Initial process priority           */  
#define INITRET      userret /* Address to which process returns   */  
  
/* Inline code to check process ID (assumes interrupts are disabled) */  
  
#define isbadpid(x)  ( ((pid32)(x) < 0) || \  
      ((pid32)(x) >= NPROC) || \  
      (proctab[(x)].prstate == PR_FREE))
```

- Mengubah NPROC dari 8 ke 150

```

Activities  Terminal  Fri 1 May, 17:34
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help
GNU nano 2.2.6 File: /home/xinu/xinu/include/process.h

/* process.h - isbadpid */

/* Maximum number of processes in the system */

#ifdef NPROC
#define NPROC 8
#endif

/* Process state constants */

#define PR_FREE 0 /* Process table entry is unused */
#define PR_CURR 1 /* Process is currently running */
#define PR_READY 2 /* Process is on ready queue */
#define PR_RECV 3 /* Process waiting for message */
#define PR_SLEEP 4 /* Process is sleeping */
#define PR_SUSP 5 /* Process is suspended */
#define PR_WAIT 6 /* Process is on semaphore queue */
#define PR_RECTIM 7 /* Process is receiving with timeout */

/* Miscellaneous process definitions */

#define PNMLEN 16 /* Length of process "name" */
#define NULLPROC 0 /* ID of the null process */

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell

```

```

    umsg32 prmsg; /* Message sent to this process */
    bool8 prhasmsg; /* Nonzero iff msg is valid */
    int16 prdesc[NDESC]; /* Device descriptors for process */
};

/* Marker for the top of a process stack (used to help detect overflow) */
#define STACKMAGIC 0x0A0AAAA9

extern struct procent proctab[];
extern int32 prcount; /* Currently active processes */
extern pid32 currp; /* Currently executing process */
xinu@xinu-develop-end:~$ nano /home/xinu/xinu/include/process.h
xinu@xinu-develop-end:~$ grep -n "NPROC" ~/xinu/include/process.h
5:#ifdef NPROC
6:#define NPROC 150
34: ((pid32)(x) >= NPROC) || \
xinu@xinu-develop-end:~$

```

2. Jalankan kode main_ex1!

```
/* main.c - main*/
```

```
#include <xinu.h>
```

```
void sndA(void);
void sndB(void);
```

```
void sndB(void);
```

```
process main(void)
```

{

```

sndA() ;

```

```
sndB( ) ;
```

```
return OK;
```

}

```
void sndA(void)
```

{

```
while(1){
```

```
printf( "A\n" );
sleep( 1000 );
```

```
sleepms(1000);
```

}

}

```
void sndB(void)
```

{

```
while(1){
```

```
printf( "B\n" );
sleep( 1000 );
```

```
sleepms(1000);
```

}

}

[illegible]

3. Jalankan kode main_ex2!

```

/* main.c - main*/

#include <xinu.h>
void sndA(void);
void sndB(void);

process main(void)
{
    resume(create(sndA, 1024, 20, "process A", 0));
    resume(create(sndB, 1024, 20, "process B", 0));
    return OK;
}

void sndA(void)
{
    while(1){
        printf("A\n");
        sleepms(1000);
    }
}

void sndB(void)
{
    while(1){
        printf("B\n");
        sleepms(1000);
    }
}

```

The screenshot shows a terminal window titled "xinu@xinu-develop-end: ~/xinu/compile". The output of the Xinu boot process is as follows:

```

Xinu for Vbox -- version #142 (xinu)  Fri 1 May 21:16:02 EDT 2026
Found Intel 82545EM Ethernet NIC
MAC address is 08:00:27:9e:f4:08
4447448 bytes of free memory. Free list:
[0x0015E320 to 0x0009FFF7]
[0x00100000 to 0x005FBFFF]
116457 bytes of Xinu code.
[0x00100000 to 0x0011C6E8]
148360 bytes of data.
[0x0011FF40 to 0x001442C7]
Obtained IP address 10.0.3.15 (0x0a00030f)
A
B
A
B
A
B
A
B
A
B
A
B
A
B

```

4. Jalankan kode main_ex3!

```

/* main.c - main*/

#include <xinu.h>
void sndChar(char);

process main(void)
{
    resume(create(sndChar, 1024, 20, "Send A", 1, 'A'));
    resume(create(sndChar, 1024, 20, "Send B", 1, 'B'));
    return OK;
}

void sndChar(char c)
{
    while(1){
        printf("%c \n", c);
        sleepms(1000);
    }
}

```

```

xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help

Xinu for Vbox -- version #144 (xinu) Fri 1 May 21:27:10 EDT 2026

Found Intel 82545EM Ethernet NIC
MAC address is 08:00:27:9e:f4:08
4447448 bytes of free memory. Free list:
[0x0015E320 to 0x0009FFFF]
[0x00100000 to 0x005FBFFF]
116425 bytes of Xinu code.
[0x00100000 to 0x0011C6C8]
148360 bytes of data.
[0x0011FF40 to 0x001442C7]

Obtained IP address 10.0.3.15 (0x0a00030f)
A
B
A
B
A
B
A B
B
A
B
A
BA

```

5. Buatlah 2 proses produser dan konsumen. Produser memproduksi angka integer dari 1-1000. Konsumer mengkonsumsi integer yang diproduksi oleh produser dan menampilkannya! (Gunakan variabel global bertipe int32 bernama n yang digunakan secara bersama oleh kedua proses)

```

/* main.c - main*/

#include <xinu.h>

int32 n = 0;

void produser(void) {
    int32 i;
    for (i = 1; i <= 1000; i++) {
        n++;
    }
}

void konsumen(void) {
    int32 i;
    for (i = 1; i <= 1000; i++) {
        printf("Nilai dari n adalah %d\n", n);
    }
}

process main(void) {
    resume(create(produser, 1024, 20, "producer", 0));
    resume(create(konsumer, 1024, 20, "consumer", 0));
    return OK;
}

```

```

Activities Terminal Fri 1 May, 18:41
xinu@xinu-develop-end: ~/xinu/compile

Xinu for Vbox -- version #146 (xinu) Fri 1 May 21:39:57 EDT 2026
Found Intel 82545EM Ethernet NIC
MAC address is 08:00:27:9e:f4:08
4447448 bytes of free memory. Free list:
[0x0015E320 to 0x0009FFF7]
[0x00100000 to 0x005FBFFF]
116489 bytes of Xinu code.
[0x00100000 to 0x0011C708]
148328 bytes of data.
[0x0011FF60 to 0x001442C7]

Obtained IP address 10.0.3.15 (0x0a00030f)
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000
Nilai dari n adalah 1000

```

Berdasarkan hasil eksekusi program, nilai *n* selalu bernilai 1000 karena proses produser berhasil menyelesaikan seluruh iterasinya sebelum proses konsumen mulai berjalan. Kondisi ini disebabkan oleh tidak adanya *delay* maupun mekanisme *time sharing* yang membuat kedua proses dieksekusi secara bergantian. Walaupun hasil yang diperoleh terlihat konsisten, program tetap berpotensi mengalami *race condition* karena variabel *n* digunakan secara bersamaan tanpa mekanisme sinkronisasi. Apabila urutan penjadwalan proses berubah, nilai yang dihasilkan dapat menjadi tidak berurutan atau bahkan tidak konsisten.